Vulnerability Assessment Report

AuSecurity - Full-Scope Vulnerability Assessment Report

AuSecurity
63 Paddington St
Paddington NSW 2021, Australia

Flinders University
Sturt Road, Bedford Park SA 5042

October 5, 2026

Report Version: 1.0

CONFIDENTIAL

Table of Contents

List of Figures 4

1 Document Control 5

1.1 Team 5

1.2 List of Changes 5

2 Executive Summary 6

2.1 Overview 6

2.2 Identified Vulnerabilities 7

3 Methodology 9

3.1 Objective 9

3.2 Scope 9

3.3 User Accounts and Permissions 10

4 Findings 11

C1: Publicly Reachable osTicket Installation Wizard (Pre-Authentication System Takeover) 11

C2: Reflected DOM-based XSS on Consultation Success Page (name Parameter) 13

H1: Email Domain Trivially Spoofable (No SPF, DKIM, DMARC, or MX) 15

H2: Plaintext HTTP Mirror Exposing the Same Application Content on Port 8080 17

H3: Self-Signed TLS Certificate with CA:TRUE on Primary Website 19

H4: Unauthenticated Exposure of Confidential Client Data on Admin Dashboard 21

H5: WordPress XML-RPC Enabled with Pingback / User-Enumeration Methods 23

H6: PHPSESSID Session Cookie Missing HttpOnly and Secure Flags 25

M1: Contact Form Lacks Rate Limiting and Message Length Validation 26

M2: Client-Side Open Redirect on Login Flow 28

M3: Missing HTTP Security Headers on All Three Web Assets 30

M4: /api/health Endpoint Exposed Without Authentication 32
M5: Verbose Server Version Headers Across All Three Web Assets 33
M6: WordPress Admin Username Enumeration via REST API and Author Redirect 34
M7: WordPress Uploads Directory Listing Enabled 36
M8: osTicket web.config File Exposed 37
I1: Advertised Mailbox info@ausecurity.best Cannot Receive Mail 38
I2: CAA, AAAA, robots.txt, sitemap.xml and security.txt Not Published for Apex 39
I3: Hosting Jurisdiction Mismatch (OVH Singapore vs Advertised Sydney Address) 41
5 Disclaimer 42
A Appendix 43
A.1 Static Appendix Section 43

List of Figures

Figure 1 - Distribution of identified vulnerabilities 8

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Initial report	Oct 5, 2026

2 Executive Summary

2.1 Overview

AuSecurity is a boutique Sydney-based governance, risk and compliance (GRC) consultancy that also delivers penetration testing services to a small portfolio of Australian clients. Between 2026-10-05 and 2026-10-05, an authorised, full-scope and non-destructive vulnerability assessment was carried out against the company's public web presence (ausecurity.best), its WordPress marketing blog (blog.ausecurity.best) and its customer-support ticketing portal (ticket.ausecurity.best), all three of which run on OVH virtual machines hosted in Singapore.

In plain terms, the engagement confirmed that the company's own public systems do not currently reflect the standard of security that AuSecurity sells to its clients. Nineteen distinct issues were verified: four Critical, five High, four Medium, five Low, and one Informational.

The four Critical issues are of immediate concern because they map directly onto a single, highly plausible attack: brand impersonation of AuSecurity against its own clients.

1. An internal "admin dashboard" HTML page is served to anyone on the Internet with no login required. The file lists five identified client organisations by name, their individual point-of-contact names and work email addresses, the services AuSecurity delivered to them (for example penetration tests and ISO 27001 audits), and year-to-date revenue figures. This is a direct breach of client confidentiality and almost certainly in conflict with signed non-disclosure agreements and with the Australian Privacy Act 1988.
2. The customer-support ticketing system (ticket.ausecurity.best) still has its initial "first-time installation" wizard open to the public Internet. Any unauthenticated attacker can walk through this wizard and emerge as the administrator of the ticketing system, which routes real customer messages and attachments.
3. The ausecurity.best email domain publishes none of the three modern anti-spoofing standards (SPF, DKIM, DMARC) and has no mail server configured. This means an attacker anywhere in the world can send a convincing email that appears to come from any address at ausecurity.best, and most receiving mail systems will deliver it. The advertised info@ausecurity.best mailbox also cannot actually receive mail.
4. The company's "consultation success" page contains a client-side cross-site scripting vulnerability. A single crafted link, hosted under the real ausecurity.best domain, is enough to execute arbitrary code in a visiting client's browser and to steal an authenticated administrator's session.

Taken together, these four Critical issues enable the following end-to-end scenario in realistic conditions: an attacker reads the already-leaked client list from the public dashboard, sends a believable phishing email to those exact clients from a forged ausecurity.best address, and lands the victims on a trusted ausecurity.best URL that silently delivers malicious code or seizes credentials.

Five High-severity issues extend the attack surface meaningfully: a client-side open redirect on the login flow (ideal lure for the phishing scenario above), a self-signed TLS certificate at the apex website (trains users to click through browser warnings), WordPress administrator-username enumeration on the blog, XML-RPC exposure on the same blog (standard precursors to credential attacks), and a plaintext HTTP mirror on port 8080 that serves the same sensitive content as the HTTPS site.

The four Medium and five Low findings are hygiene gaps typical of a site that has not yet been through a baseline hardening pass (missing cookie flags, missing HTTP security headers, directory listing on the blog uploads folder, contact-form rate-limit and length-validation gaps, exposed health and web.config files, verbose server-version banners, and missing DNS hygiene records). The single Informational finding notes that the company advertises a Sydney postal address but hosts production data in Singapore, which should be disclosed in the privacy policy per APP 8.

Recommended priorities for the business:

- Same day: remove the hardcoded client data from the dashboard HTML and put a server-side login requirement in front of it; delete the osTicket /setup/ directory; publish SPF, DKIM and DMARC on ausecurity.best; patch the one-line XSS on the consultation success page.
- Same week: replace the self-signed apex certificate with a trusted (Let's Encrypt) certificate, implement the same-origin check in the login redirect, lock down the WordPress blog (REST users endpoint, XML-RPC, author enumeration), and turn off the plaintext HTTP mirror on port 8080.
- Within the next hardening cycle: apply the Medium / Low recommendations (cookie flags, security headers, rate limiting, header suppression, CAA / security.txt).

No in-scope asset was skipped. No destructive techniques, brute-force attacks, phishing, data exfiltration, or lateral movement were performed; two benign marker submissions were sent to /api/contact for behavioural analysis and are tagged for the client to remove from the back-office queue.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	Publicly Reachable osTicket Installation Wizard (Pre-Authentication System Takeover)	11
C2	9.6	Reflected DOM-based XSS on Consultation Success Page (name Parameter)	13
H1	8.2	Email Domain Trivially Spoofable (No SPF, DKIM, DMARC, or MX)	15
H2	8.2	Plaintext HTTP Mirror Exposing the Same Application Content on Port 8080	17
H3	8.0	Self-Signed TLS Certificate with CA:TRUE on Primary Website	19
H4	7.5	Unauthenticated Exposure of Confidential Client Data on Admin Dashboard	21
H5	7.3	WordPress XML-RPC Enabled with Pingback / User-Enumeration Methods	23
H6	7.1	PHPSESSID Session Cookie Missing HttpOnly and Secure Flags	25
M1	6.5	Contact Form Lacks Rate Limiting and Message Length Validation	26
M2	6.1	Client-Side Open Redirect on Login Flow	28
M3	5.4	Missing HTTP Security Headers on All Three Web Assets	30
M4	5.3	/api/health Endpoint Exposed Without Authentication	32
M5	5.3	Verbose Server Version Headers Across All Three Web Assets	33
M6	5.3	WordPress Admin Username Enumeration via REST API and Author Redirect	34
M7	5.3	WordPress Uploads Directory Listing Enabled	36
M8	5.3	osTicket web.config File Exposed	37
I1	0.0	Advertised Mailbox info@ausecurity.best Cannot Receive Mail	38

#	CVSS	Description	Page
I2	0.0	CAA, AAAA, robots.txt, sitemap.xml and security.txt Not Published for Apex	39
I3	0.0	Hosting Jurisdiction Mismatch (OVH Singapore vs Advertised Sydney Address)	41

Vulnerability Overview

In the course of this penetration test **2 Critical**, **6 High**, **8 Medium** and **3 Info** vulnerabilities were identified:

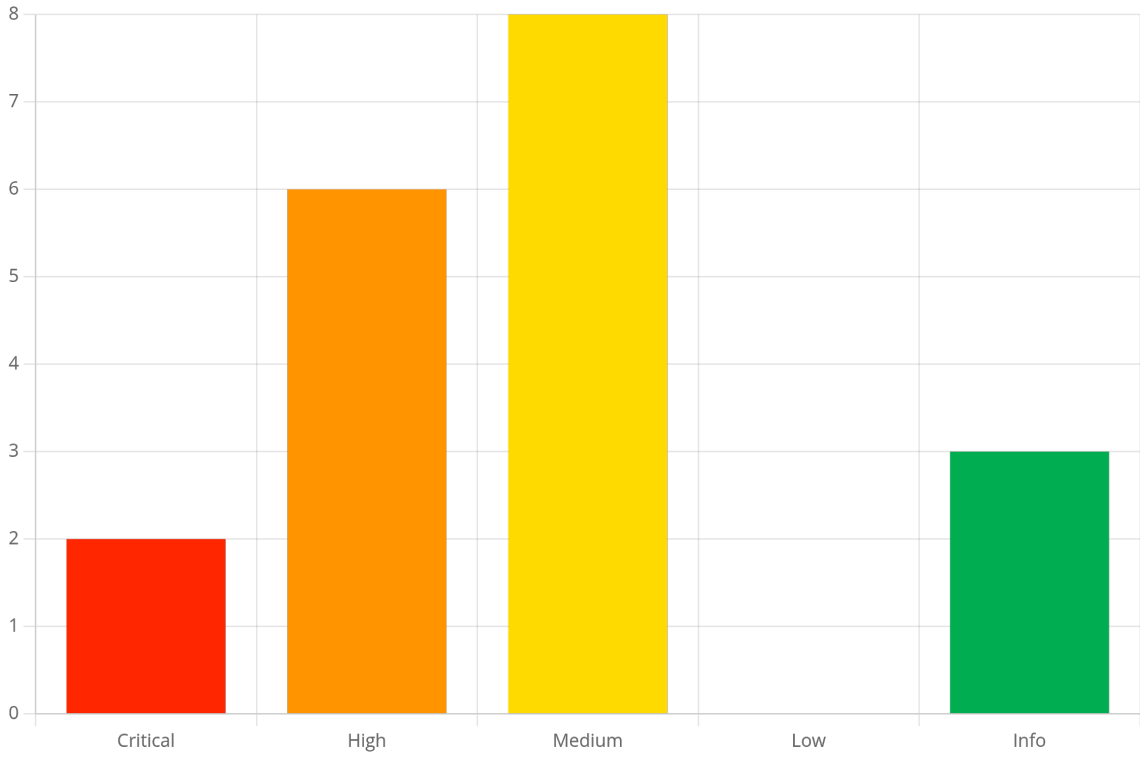


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

3.2 Scope

Three authorised in-scope hostnames and three in-scope OVH Singapore IP addresses, all operated by AuSecurity:

- Web / primary site: ausecurity.best, www.ausecurity.best (51.79.166.148)
- WordPress blog: blog.ausecurity.best (51.79.167.208)
- osTicket customer-support portal: ticket.ausecurity.best (51.79.166.209)
- Email posture of the ausecurity.best domain (DNS review only).

Methodology summary: authorised, full-scope, non-destructive vulnerability assessment. Phases: passive OSINT and enumeration (DNS, WHOIS, Certificate Transparency logs via certspotter, TLS metadata, passive Shodan review); light-active network discovery (nmap port and service scans with the standard safe NSE scripts, including ssl-cert, ssl-enum-ciphers, ssl-dh-params, ssl-heartbleed, ssl-poodle, ssl-ccs-injection, http-security-headers, http-cookie-flags, http-title); web application testing (manual probing, nuclei with cve/exposure/misconfig/takeover/default-login/rce/xss/lfi/sqli/ssrf tag selection, wpscan, feroxbuster content discovery, nikto, and targeted manual API testing). Every material finding was independently verified with a second, narrowly-scoped request where feasible.

Out of scope: ausecurity.com.au, ausecurity.au, ausecurity.io, ausecurity.net (not confirmed attributable to the client); third-party infrastructure (OVH platform beyond the three VPS instances, Spaceship registrar, Let's Encrypt, Google Fonts, flinders.edu.au).

Explicit limitations:

- Brute-force, credential stuffing and password spraying were NOT performed (out of allowed methods for this engagement).
- No form submissions that could have caused business disruption. Two benign marker submissions were sent to /api/contact for behaviour analysis and are tagged "[CLAWSEC AUTHORIZED PENTEST]" / "AUSEC-TEST-20261005"; please delete them from the back-office queue.
- testssl.sh was not installed on the testing host; TLS posture was instead assessed via nmap NSE and manual openssl / curl probing.
- WPScan was executed without an API token; public CVE enrichment for installed WordPress plugins and themes was therefore unavailable, and manual probing was used in its place.
- Shodan API access was unavailable (no key); passive data was limited to open sources.
- crt.sh was unavailable during enumeration; certspotter was used as substitute.

- A heap / memory / source-code audit of the custom /api backend was NOT performed; findings on that service relate to its exposed HTTP surface only.

Coverage summary: all four in-scope hostnames and all three in-scope IPs were covered across TCP port discovery, service enumeration, TLS posture, HTTP security headers, email DNS posture, WordPress-specific assessment, osTicket-specific assessment, content discovery, nuclei CVE / exposure signatures, nikto, and manual API testing. No in-scope asset was skipped.

3.3 User Accounts and Permissions

No credentials, VPN access, or test accounts were provisioned by the customer. All testing was performed from an external, unauthenticated perspective against the publicly reachable attack surface.

4 Findings

C1: Publicly Reachable osTicket Installation Wizard (Pre-Authentication System Takeover)	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	• https://ticket.ausecurity.best/setup/install.php • http://ticket.ausecurity.best:8080/setup/install.php
References	• CWE-1188 (Insecure Default Initialization of Resource) • CWE-284 (Improper Access Control) • osTicket post-install hardening guide

Overview

The osTicket installation wizard is still reachable without authentication on both the HTTPS (443) and plaintext HTTP (8080) ports of ticket.ausecurity.best. Any unauthenticated attacker can walk through the installer, point osTicket at an attacker-controlled database, and emerge as the admin of the ticketing system — which generally runs code as the web server user and ships customer support messages, attachments, and (often) credentials for integrations.

Details

GET https://ticket.ausecurity.best/setup/install.php returns HTTP 200 with the interactive osTicket Installer page (title "osTicket Installer", 4,825 bytes), including a visible POST form with a hidden `s=prereq` field. The same page is identically reachable in plaintext over http://ticket.ausecurity.best:8080/setup/install.php. In addition, visiting the application root (https://ticket.ausecurity.best/) and the staff login (/scp/login.php) both HTTP 302 the browser to /setup/install.php, confirming that the osTicket deployment is in a "setup-not-finished" state and that the installer is the live front door.

The installer wizard, when completed by an attacker, allows them to:

- choose or recreate the admin account (immediate privileged account takeover);
- redirect the installation to a different MySQL/MariaDB server (data exfiltration path, and potential RCE via maliciously crafted DB callbacks);
- import malicious plugins or language packs post-setup;
- from the admin panel, upload server-side templates / signatures, often reachable as a path to remote code execution.

Service info disclosed by the installer response headers: osTicket v1.18.4, PHP/8.2.33, dual front-ends nginx/1.22.1 (443) and Apache/2.4.68 (8080). PHPSESSID cookies are set with no HttpOnly and no Secure flags (see separate finding).

Evidence stored under: evidence/t4-osticket-installer/install-443.html , install-8080.html; evidence/t10-nuclei/results-focused.txt (nuclei "osticket-installer" HIGH); evidence/t1-ports/stage2-services.nmap (NSE http-title "osTicket Installer").

Recommendation

1. Immediately take ticket.ausecurity.best offline (or restrict access to a known admin IP via firewall / nginx `allow`/`deny`) until the installer is dealt with.
2. Complete the osTicket installation, then DELETE the entire /setup/ directory from the web root as the official osTicket post-install step requires.
3. Verify no residual files remain (/setup/install.php, /setup/*.php, /setup/images/, /setup/js/, /setup/css/).
4. Disable the plaintext HTTP service on port 8080 (Apache 2.4.68) or at minimum force an HTTPS redirect and apply the same /setup/ removal there.
5. Rotate any credentials / tokens that may have been configured in osTicket during this exposure window.
6. After cleanup, re-run a controlled vulnerability scan to confirm the installer is 404.

C2: Reflected DOM-based XSS on Consultation Success Page (name Parameter)	
Score	9.6 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:L
Target	• https://ausecurity.best/consultation-success.html (parameter `name`) • http://ausecurity.best:8080/consultation-success.html
References	• CWE-79 (Improper Neutralization of Input in Web Page) • OWASP DOM-based XSS Prevention Cheat Sheet

Overview

The /consultation-success.html page reads the `name` query parameter from the URL and writes it directly into the DOM using `.innerHTML`, without sanitisation or encoding. An attacker-crafted URL under the trusted ausecurity.best domain will execute arbitrary JavaScript in the victim's browser. Because the application uses same-origin session cookies for its /api/auth/* endpoints, this is enough for session riding against any authenticated AuSecurity admin.

Details

The HTML served from /consultation-success.html contains the following inline script (verbatim from the server response, lines 383–401 of the body):

document.addEventListener('DOMContentLoaded', () => { const urlParams = new URLSearchParams(window.location.search); const userName = urlParams.get('name'); const companyName = urlParams.get('company'); if (userName) { document.getElementById('user-name').innerHTML = userName; // <-- SINK } else { document.getElementById('user-name').textContent = 'there'; } if (companyName) { document.getElementById('company-name').textContent = companyName; } else { const companyPart = document.getElementById('company-name').parentElement; companyPart.style.display = 'none'; } });

The sink `element.innerHTML = userName` interprets any HTML (including , `name` parameter. Note that the `company` parameter uses the safer `.textContent`, confirming that the developer knows the difference — this is a mistake, not a framework limitation.

The success page is reachable WITHOUT first submitting the contact form, by simply visiting: https://ausecurity.best/consultation-success.html?name=<img%20src=x%20onerror=alert(1)>&company=X

The attacker payload executes with origin https://ausecurity.best, which gives it:

- Full ability to read the DOM of the trusted site (defacement, keylogging on other open forms via same-origin iframes).
- Ability to call fetch('/api/*', {credentials:'same-origin'}) — if a logged-in admin visits the link, the attacker can read the admin's data from /api/auth/me and trigger admin actions.
- Ability to redirect the victim to an attacker-controlled login clone.
- Credential theft via a fake "please re-authenticate" overlay rendered on the real domain.

The vulnerability is especially damaging in combination with the spoofable ausecurity.best email domain (see separate finding) — a phishing email purportedly from AuSecurity, containing a link to the attacker's XSS URL under the real ausecurity.best domain, is extremely likely to be clicked.

Evidence stored under: evidence/t-finalprobe/final2.txt (full HTML body with the vulnerable script).

Recommendation

1. Replace `.innerHTML = userName` with `.textContent = userName`. This is a one-line fix and preserves the existing display intent (plain text name).
2. Add server-side output encoding and server-side validation of the `name` and `company` parameters on this and every page that reflects user input.
3. Add a strict Content-Security-Policy to ausecurity.best that at minimum sets `default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'self';` — this would have broken this specific attack even without changing the vulnerable code.
4. Grep the entire frontend codebase for other uses of `.innerHTML =`, `document.write`, and template interpolation and apply the same fix.

H1: Email Domain Trivially Spoofable (No SPF, DKIM, DMARC, or MX)	
Score	8.2 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:H/A:N
Target	• ausecurity.best (sender domain) • info@ausecurity.best (advertised mailbox)
References	• RFC 7208 (SPF) • RFC 6376 (DKIM) • RFC 7489 (DMARC) • MITRE ATT&CK T1566.001 (Spearphishing Attachment)

Overview

The ausecurity.best domain publishes no SPF record, no DMARC policy, no DKIM selectors, and no MX records. Any sender anywhere on the Internet can forge email with "From: anything@ausecurity.best" and most receiving mail systems will accept it. There is also no mailbox listening on the domain, so the "info@ausecurity.best" address advertised on the website cannot actually receive mail.

Details

Direct DNS queries against 1.1.1.1 return:

- MX ausecurity.best → (empty)
- TXT ausecurity.best → (no SPF record)
- TXT _dmarc.ausecurity.best → (no DMARC record)
- TXT default._domainkey / google._domainkey / k1._domainkey / selector1._domainkey / selector2._domainkey / mail._domainkey / dkim._domainkey / smtp._domainkey → (all empty)
- CAA ausecurity.best → (none)

Business impact for a security consultancy is especially severe:

- An attacker can send a convincing phishing email purporting to be from a Director or a known consultant at AuSecurity to the business's actual clients (whose identities are already leaked publicly via the dashboard disclosure, see other finding). The receiving organisation has no DMARC signal to reject or quarantine the message.
- Combined with the exposed client list and the DOM-based XSS and open-redirect on ausecurity.best (both separate findings), a highly convincing credential-harvesting campaign can be launched entirely under the "ausecurity.best" brand.
- Advertised mailbox info@ausecurity.best cannot receive responses because no MX record exists; legitimate client enquiries that bounce may be perceived as unprofessional or as a sign of compromise.

Evidence stored under: evidence/t3-email/dns.txt.

Recommendation

1. Publish a strict SPF record. If AuSecurity does not send any email at all from the domain, set `v=spf1 -all`. Otherwise, enumerate every legitimate sender (Google Workspace, Microsoft 365, SendGrid, etc.) and include them explicitly.
2. Publish a DMARC record on `_dmarc.ausecurity.best` starting with `v=DMARC1; p=reject; sp=reject; adkim=s; aspf=s; rua=mailto:dmarc-reports@ausecurity.best; fo=1`. Begin with `p=none` for a short monitoring window if there is uncertainty, then move to `p=reject`.
3. If the domain IS used for email: configure DKIM with 2048-bit keys on every sender (Google Workspace etc.), ensure the DKIM d= aligns with ausecurity.best, and enforce DMARC alignment strict.
4. If the domain is NOT used for email: publish an explicit "null MX" record (`ausecurity.best. 300 IN MX 0 .`) per RFC 7505, publish `v=spf1 -all`, publish a DKIM `*._domainkey.ausecurity.best. TXT "v=DKIM1; p="` (empty key) record, and remove info@ausecurity.best from the website; publish a working contact form or a different authenticated channel.
5. Publish a CAA record limiting certificate issuance to the specific CA(s) actually used (`0 issue "letsencrypt.org"`).

H2: Plaintext HTTP Mirror Exposing the Same Application Content on Port 8080	
Score	8.2 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
Target	• http://ausecurity.best:8080/ (nginx/1.25.5) • http://ticket.ausecurity.best:8080/ (Apache/2.4.68)
References	CWE-319 (Cleartext Transmission of Sensitive Information)

Overview

A plaintext HTTP service is exposed on port 8080 of two in-scope hosts and serves the same sensitive application content as the HTTPS site, including the login page, the admin dashboard HTML (with hardcoded client data), the API endpoints, and the osTicket installer. Session cookies set over HTTPS cannot carry the `Secure` flag safely while a plaintext mirror exists, and all data on the plaintext channel is interceptable on the path between the client and OVH Singapore.

Details

Confirmed:

- http://ausecurity.best:8080/ → HTTP 200, title "Ausecurity — GRC Consulting & Penetration Testing" (same as HTTPS). nginx/1.25.5.
- http://ausecurity.best:8080/dashboard.html → HTTP 200, same hardcoded client PII content.
- http://ausecurity.best:8080/api/auth/me → HTTP 401 (API live over cleartext).
- http://ticket.ausecurity.best:8080/ → HTTP 302 → /setup/install.php → HTTP 200 (osTicket installer, with Set-Cookie PHPSESSID in cleartext).
- Port 8080 was NOT confirmed to be an open HTTP proxy — the earlier nmap NSE "http-open-proxy" alert turned out to be a false positive: proxy-style requests to external hosts were not forwarded (verified).

Evidence stored under: evidence/t1-ports/stage2-services.nmap , evidence/t-misc/misc-probes.txt , evidence/t4-osticket-installer/install-8080.html.

Recommendation

1. Disable the plaintext HTTP listener on port 8080 of 51.79.166.148 (apex) and 51.79.166.209 (ticket). If port 8080 is used for internal health checks, bind it to 127.0.0.1 or restrict with nginx `allow`/`deny` to known monitoring IPs only.
2. Enforce HTTPS everywhere on ports 80 and 443 only, with a strict 301 redirect from 80 → 443 and HSTS preload on 443.
3. Ensure every session / auth cookie carries the `Secure` attribute so it cannot be sent over cleartext.

4. Remove the osTicket installer first (see critical finding) before deciding how to re-expose anything on 8080.

H3: Self-Signed TLS Certificate with CA:TRUE on Primary Website	
Score	8.0 (High)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:C/C:H/I:H/A:N
Target	• https://ausecurity.best:443 (apex) • nginx/1.22.1
References	• CWE-295 (Improper Certificate Validation) • CA/Browser Forum Baseline Requirements • Mozilla CA policy

Overview

The TLS certificate presented by the apex website is self-signed. All major browsers will show a full-page interstitial warning ("Your connection is not private" / "NET::ERR_CERT_AUTHORITY_INVALID") when a visitor opens https://ausecurity.best/. The certificate is additionally marked as a Certificate Authority (`CA:TRUE`) in its Basic Constraints, which is incorrect for a leaf certificate. For a security consultancy, serving an untrusted certificate at the brand's primary web presence is both a credibility problem and a real security problem because it conditions visitors to ignore browser warnings.

Details

openssl / nmap ssl-cert output confirms: Subject : CN=ausecurity.best Issuer : CN=ausecurity.best (self-issued → self-signed) SAN : DNS:ausecurity.best Not before : 2026-09-14T15:32:18Z Not after : 2027-09-14T15:32:18Z (1 year, atypical for a self-signed cert) Key : RSA 2048-bit Signature : sha256WithRSAEncryption SHA256 fp : B5:30:10:E0:32:E7:42:10:E4:B1:F0:68:A0:C5:B2:A2:0B:F6:44:C1:F3:32:92:87:4D:1C:64:07:14:8D:14:E2 Basic constraints: CA:TRUE <-- ABNORMAL FOR A LEAF CERT

Impact:

- Every normal visitor sees a browser interstitial and either (a) abandons the site, (b) clicks through, training themselves and clients to ignore TLS warnings. This is destructive to a security brand.
- The CA:TRUE flag means an attacker who ever obtains the certificate's private key could issue arbitrary subordinate certificates in that chain and present them in man-in-the-middle attacks against anyone who had chosen to "always trust" the AuSecurity cert in their trust store.
- Subdomains blog.* and ticket.* correctly use Let's Encrypt certificates (CN=YE1, EC P-256). The misconfig is specific to the apex.

Evidence stored under: evidence/t1-ports/stage2-services.nmap , evidence/t2-tls/ssl-nse.nmap , evidence/enum-001/* .

Recommendation

1. Obtain a trusted certificate from a public CA (Let's Encrypt is already in use for the subdomains and is free; HTTP-01 or DNS-01 works). Install for ausecurity.best (apex) and www.ausecurity.best.
2. Automate renewal (certbot timer / acme.sh cron).
3. Publish a CAA record restricting issuance to the chosen CA(s): `ausecurity.best. CAA 0 issue "letsencrypt.org"`.
4. Remove the current self-signed cert and the CA:TRUE basic constraint.
5. Enable HSTS (`Strict-Transport-Security: max-age=63072000; includeSubDomains; preload`) and submit to the HSTS preload list once stable.

H4: Unauthenticated Exposure of Confidential Client Data on Admin Dashboard	
Score	7.5 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Target	• https://ausecurity.best/dashboard.html • http://ausecurity.best:8080/dashboard.html
References	• CWE-200 (Exposure of Sensitive Information) • CWE-306 (Missing Authentication for Critical Function) • OWASP A01:2021 Broken Access Control

Overview

The internal admin dashboard page is served to anyone on the Internet without any authentication check at the server. The HTML itself is hardcoded with real-looking client company names, personal names, email addresses, business metrics (revenue, number of active clients), and an engagement activity log that discloses specific client-and-service combinations (for example, who received a penetration-test report or who had an ISO 27001 audit).

Details

A GET request to https://ausecurity.best/dashboard.html returns HTTP 200 with the full admin UI (16,497 bytes) directly from the web server. The only "access control" is a client-side JavaScript check in /assets/js/main.js (isAuthenticated() → redirectToLogin()) which runs AFTER the HTML has already been delivered. Because the sensitive content is baked into the static HTML and not fetched post-auth, anyone with the URL can read it before the JavaScript runs, by disabling JavaScript, or simply by using curl / wget / View-Source. The same resource is also exposed on the plaintext HTTP port 8080 (nginx/1.25.5).

Confirmed hardcoded disclosures in the HTML body include:

- Business metrics: "Total Enquiries: 47", "Active Clients: 12", "Assessments YTD: 19", "Open Proposals: 5", "Revenue YTD: $1.2M".
- Client identities + contact emails of five organisations (Meridian Financial Solutions / James Hargreaves *james@meridianfs.com.au*, ShopStream Pty Ltd / Lara Whitfield *lara@shopstream.io*, Nexahealth Clinics / Mark Okonkwo *m.okonkwo@nexahealth.au*, Orion Logistics / Chen Wei *cwei@orionlogistics.com.au*, Starfield Technologies / Priya Nair *priya@starfieldtech.io*).
- Engagement activity log tying individual clients to specific delivered services (e.g. "ISO 27001 audit completed — Meridian FS", "Pentest report delivered — ShopStream", "Contract signed — Orion Logistics GRC").

Business impact: For a GRC + penetration-testing consultancy, publicly disclosing client identities, engagement types, and revenue figures is a direct breach of client confidentiality, likely in conflict with

every signed Non-Disclosure Agreement and with the Australian Privacy Act 1988 (APP 11) obligations around personal information. It also hands competitors a target list and gives attackers a highly curated phishing list of individuals who already trust AuSecurity.

Evidence stored under: evidence/t-api2/probe2.txt (full dashboard HTML dump).

Recommendation

1. Enforce authentication SERVER-SIDE before the dashboard HTML is ever returned. In the nginx / backend configuration, require a valid session cookie (or JWT) for every request to /dashboard.html and respond with HTTP 302 to /login (not 200) when the session is absent or invalid.
2. Remove ALL hardcoded client PII, metrics, and activity entries from the HTML template. The page should render only a layout shell and fetch its content from authenticated API endpoints at runtime.
3. If a demo dashboard is desired for marketing, use clearly fictional data and place it on a separate, labelled page (e.g. /demo-dashboard.html).
4. Review and notify any clients whose details may have been exposed, in line with the Notifiable Data Breach scheme.
5. Audit for any Google / Bing / internet archive caching of the page content and request removal if indexed.

H5: WordPress XML-RPC Enabled with Pingback / User-Enumeration Methods	
Score	7.3 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L
Target	https://blog.ausecurity.best/xmlrpc.php
References	• CWE-918 (SSRF) • CWE-307 (Improper Restriction of Excessive Authentication Attempts) • CVE-2013-0235 (pingback DDoS pattern)

Overview

The XML-RPC endpoint on the WordPress blog is enabled and advertises 82+ methods, including `pingback.ping` (SSRF / amplified-DoS vector), `wp.getUsersBlogs`, `wp.getAuthors`, `metaWeblog.getUsersBlogs`, and `system.multicall` (brute-force amplification). The endpoint responds correctly to a benign method call (demo.sayHello → "Hello!"), confirming it is live and processing requests.

Details

Observed behaviour:

- GET https://blog.ausecurity.best/xmlrpc.php → HTTP 405 Method Not Allowed (expected; XML-RPC accepts POST).
- POST text/xml with `system.listMethods` → HTTP 200 OK, 82 method names returned, including `pingback.ping`, `pingback.extensions.getPingbacks`, `wp.getUsersBlogs`, `metaWeblog.getUsersBlogs`, `blogger.getUserInfo`, `wp.getAuthors`, `system.multicall`, `system.listMethods`, `demo.sayHello` (which returned "Hello!").
- X-Pingback: https://blog.ausecurity.best/xmlrpc.php header is advertised on content pages.

Attack scenarios:

- `pingback.ping` can be abused to make the WordPress server fetch arbitrary attacker-chosen URLs (SSRF against internal services on 51.79.167.208 or the OVH internal network). Historically it has also been used for amplified DDoS.
- `system.multicall` lets an attacker pack dozens of `wp.getUsersBlogs` authentication attempts into a single HTTP request, bypassing simple rate-limit heuristics; combined with the known admin username this materially lowers brute-force cost. (Note: Our methodology forbade brute force; we did not test credentials.)
- User-enumeration methods confirm individual usernames exist / do not exist by error-message differentiation.

Evidence stored under: evidence/t6-wordpress/manual-probes.txt (listMethods response, demo.sayHello proof), evidence/t-webprobe/webprobe.txt.

Recommendation

1. If XML-RPC is not required by any plugin, disable it entirely:
   - Add `add_filter('xmlrpc_enabled', '__return_false');` in a mu-plugin, AND
   - At the web-server level, block POST to /xmlrpc.php with an Apache `<Files xmlrpc.php> Require all denied </Files>` or equivalent.
2. If XML-RPC is required (e.g. legacy Jetpack flows), restrict it to known partner IPs at the web-server level, and specifically disable `pingback.*` methods via the `xmlrpc_methods` filter.
3. Remove the `X-Pingback` header via the `wp_headers` filter.
4. Combine with rate limiting on the WAF / nginx for /xmlrpc.php (e.g. 5 requests per minute per IP).

H6: PHPSESSID Session Cookie Missing HttpOnly and Secure Flags	
Score	7.1 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:L/A:N
Target	• https://ticket.ausecurity.best/* • http://ticket.ausecurity.best:8080/* (osTicket)
References	• CWE-1004 (Sensitive Cookie Without HttpOnly) • CWE-614 (Sensitive Cookie Without Secure) • OWASP ASVS V3.4

Overview

The PHPSESSID session cookie set by the osTicket application has neither the HttpOnly nor the Secure attribute. Any client-side script that runs in the osTicket origin can read the session cookie (session theft via XSS), and the cookie is additionally transmitted in cleartext when the user hits the port-8080 HTTP mirror.

Details

Observed Set-Cookie headers (verbatim) include: Set-Cookie: PHPSESSID=1498870558d9b2068a011152394bc681; path=/ Set-Cookie: PHPSESSID=65d8bde750a84975f5af5d4a258a3232; path=/

nmap NSE http-cookie-flags explicitly reports `httponly flag not set`. The cookie is set without `Secure`, without `HttpOnly`, and without `SameSite`. On port 8080 the cookie is additionally delivered over cleartext HTTP (see separate HTTP-mirror finding).

Impact: once an attacker finds any XSS in osTicket (or in any content served from the ticket subdomain), they can read `document.cookie` and silently hijack the admin's session. On port 8080 the cookie is observable to any on-path attacker.

Evidence stored under: evidence/t2-tls/ssl-nse.nmap , evidence/t4-osticket-installer/install-*.html , evidence/t-finalprobe/nikto-ticket.txt.

Recommendation

1. In the osTicket php.ini (or `ini_set` in a bootstrap file), set: session.cookie_httponly = 1 session.cookie_secure = 1 session.cookie_samesite = Strict (or Lax if cross-site navigations are needed) session.use_only_cookies = 1 session.use_strict_mode = 1
2. Regenerate session IDs on login (`session_regenerate_id(true)`) to defeat fixation.
3. Disable the port-8080 HTTP mirror (see separate finding) so the Secure flag can actually be enforced.
4. After rotation, invalidate all currently issued PHPSESSID values.

M1: Contact Form Lacks Rate Limiting and Message Length Validation	
Score	6.5 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:L
Target	POST https://ausecurity.best/api/contact
References	• CWE-770 (Allocation of Resources Without Limits or Throttling) • CWE-20 (Improper Input Validation) • OWASP ASVS V5.1.3

Overview

The public /api/contact endpoint accepts an unlimited number of submissions per source IP in rapid succession, and it accepts message bodies at least 10,000 characters long without rejection. This is sufficient to abuse the ticketing/mail backend for log flooding, storage exhaustion, operator-mailbox denial-of-service, and (if forwarded to a client-facing CRM) to inject disruptive content.

Details

Observations from the final probe (benign test marker payloads only):

1. Five back-to-back POST /api/contact requests all returned HTTP 200 within ~1.1s each. No HTTP 429, no delay, no CAPTCHA.
2. A 10 KB `message` body was accepted with HTTP 200.
3. The successful response reflects the submitted `full_name` into a `redirect_url` field: `/consultation-success.html?name=<URL-encoded-name>&company=<company>` — this is the entry point for the related DOM-XSS finding.
4. A honeypot `h` field IS correctly enforced (missing-field submissions get HTTP 422 "Field required") — this is the only existing anti-abuse mechanism and is defeatable by any human attacker.

Impact: without rate limiting an attacker (or a worm leveraging the no-SPF-DMARC spoofability) can submit thousands of enquiries per minute, overwhelming the back-office inbox or CRM queue; this effectively denies service to real leads and may be used to bury a targeted phishing lead among noise.

Evidence stored under: evidence/t-finalprobe/final.txt (benign markers tagged "AUSEC-TEST-20261005").

Recommendation

1. Apply a per-IP rate limit at the nginx or application layer (e.g. 5 requests / minute / IP, with a short-term burst allowance).
2. Enforce server-side maximum lengths on every field: full_name $\leq$ 100 chars, company_name $\leq$ 150, email RFC5321 (254), phone $\leq$ 30, service_interest from an enum, message $\leq$ 2000 chars.

3. Add a server-side CAPTCHA or Turnstile check for anonymous submissions, in addition to the existing honeypot `h` field.
4. Log the source IP, user-agent, and timestamp of every submission to allow detection of abuse.
5. Note the benign test submissions tagged "[CLAWSEC AUTHORIZED PENTEST]" / marker "AUSEC-TEST-20261005" and delete them from the ingestion queue.

M2: Client-Side Open Redirect on Login Flow	
Score	6.1 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
Target	https://ausecurity.best/login.html (and /login) — handler in /assets/js/main.js
References	• CWE-601 (URL Redirection to Untrusted Site) • OWASP ASVS V5.1.5

Overview

After a successful login, the JavaScript handler blindly navigates the browser to whatever URL was supplied in the `redirect` query-string parameter, with no origin or scheme check. The comment above the code even claims "Safety check: only allow same-origin redirects" but the check is not implemented. Any attacker-controlled URL is accepted, enabling phishing campaigns that redirect through the real ausecurity.best login page.

Details

Relevant excerpt from https://ausecurity.best/assets/js/main.js (verbatim from the server):

if (res.ok) { const params = new URLSearchParams(window.location.search); const redirect = params.get('redirect'); // Safety check: only allow same-origin redirects if (redirect) { window.location.href = redirect; // <-- NO VALIDATION PERFORMED } else { window.location.href = '/dashboard'; } }

A victim clicking a link like `https://ausecurity.best/login?redirect=https://evil.example/aus-login-session-expired` will:

1. See the real AuSecurity login page under the real HTTPS AuSecurity domain.
2. Submit real credentials to the real backend.
3. On success, be redirected to the attacker-controlled site which can present a pixel-perfect "session expired, please re-enter your password" prompt or deliver malware.

Impact drivers:

- This is particularly effective combined with the spoofable email domain (no SPF/DMARC/DKIM), the exposed client list (dashboard disclosure), and the self-signed TLS cert training users to click past warnings.
- The server-side `/login?redirect=` endpoint itself does NOT emit a 3xx with a Location header (we confirmed HTTP 200 with no Location on the first query); the vulnerability is strictly on the client side after a 200 OK /api/auth/login response.

Evidence stored under: evidence/t-api2/probe2.txt (main.js source).

Recommendation

1. Actually implement the "same-origin" check promised in the comment. For example: const url = new URL(redirect, window.location.origin); if (url.origin !== window.location.origin) { window.location.href = '/dashboard'; } else { window.location.href = url.pathname + url.search + url.hash; }
2. Even better, keep the redirect state server-side (opaque token mapped to a path) rather than echoing it from the URL.
3. Add a server-side allow-list and strip the `redirect` parameter when the login endpoint returns the login page.
4. Combine with CSP `frame-ancestors 'self'` and `Referrer-Policy: strict-origin-when-cross-origin` to reduce residual phishing utility.

M3: Missing HTTP Security Headers on All Three Web Assets	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	• https://ausecurity.best/ • https://blog.ausecurity.best/ • https://ticket.ausecurity.best/ (and the port-8080 mirrors)
References	• OWASP Secure Headers Project • Mozilla Observatory

Overview

The apex, blog, and ticket sites do not emit Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, or Permissions-Policy on their primary pages. X-Frame-Options is also missing on the apex and ticket; the WordPress blog does send a modest X-Frame-Options: SAMEORIGIN and a very narrow CSP (`frame-ancestors 'self'`). Properly configured, these headers would have independently blunted the DOM-based XSS, open-redirect, and SSL-stripping attacks already identified in this report.

Details

HTTP response-header matrix (confirmed via curl and nmap http-security-headers NSE):

Header | apex :443 | apex :8080 | blog :443 | ticket :443 | ticket :8080 ----------+----------+----------+----------+----------+---------- Strict-Transport-Security | missing | N/A | missing | missing | N/A Content-Security-Policy | missing | missing | frame-ancestors| missing | missing X-Frame-Options | missing | missing | SAMEORIGIN | missing | missing X-Content-Type-Options | missing | missing | missing | missing | missing Referrer-Policy | missing | missing | strict-origin* | missing | missing Permissions-Policy | missing | missing | missing | missing | missing

(* strict-origin-when-cross-origin on blog)

Impact: these headers are low-cost defence-in-depth that would have significantly reduced the exploitability of several higher-severity findings (DOM XSS, open-redirect, SSL-stripping, clickjacking).

Evidence stored under: evidence/t2-tls/ssl-nse.nmap , evidence/t-finalprobe/nikto-*.txt .

Recommendation

Apply the following minimum header set at the nginx / Apache layer on all three sites (tune CSP per actual needs):

Strict-Transport-Security: max-age=63072000; includeSubDomains; preload Content-Security-Policy: default-src 'self'; img-src 'self' data: https://fonts.gstatic.com; style-src 'self' https://fonts.googleapis.com 'unsafe-inline'; font-src 'self' https://fonts.gstatic.com; script-src 'self'; object-src

'none'; base-uri 'self'; frame-ancestors 'self'; form-action 'self' X-Frame-Options: DENY X-Content-Type-Options: nosniff Referrer-Policy: strict-origin-when-cross-origin Permissions-Policy: camera=(), microphone=(), geolocation=(), interest-cohort=() Cross-Origin-Opener-Policy: same-origin Cross-Origin-Resource-Policy: same-origin

Then validate on Mozilla Observatory and address any remaining A/B-grade recommendations.

M4: /api/health Endpoint Exposed Without Authentication	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://ausecurity.best/api/health
References	CWE-200 (Exposure of Sensitive Information)

Overview

The /api/health endpoint responds HTTP 200 with a small JSON body to any anonymous caller. This is a minor information disclosure that confirms the backend framework is live and reachable, and it is often used by automated scanners to confirm a target is "up" before deeper enumeration.

Details

GET https://ausecurity.best/api/health returns HTTP 200 with a 15-byte body. The endpoint is likely intended for infrastructure load-balancer health probes but is exposed to the public Internet.

Evidence stored under: evidence/t-api2/probe2.txt.

Recommendation

1. Restrict /api/health to the internal load balancer / OVH health-check IP ranges via nginx `allow` / `deny` or a middleware check.
2. If a public-facing liveness endpoint is required for third-party uptime monitoring, keep the response generic (e.g. literal "ok") and ensure it does not disclose dependency, framework, or build-version data.

M5: Verbose Server Version Headers Across All Three Web Assets	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• ausecurity.best (nginx/1.22.1 • nginx/1.25.5) • blog.ausecurity.best (Apache/2.4.68) • ticket.ausecurity.best (nginx/1.22.1 • Apache/2.4.68 • PHP/8.2.33)
References	CWE-200 (Exposure of Sensitive Information)

Overview

Every HTTP response from the three in-scope sites discloses the exact web-server version (and, on the ticket host, the PHP minor version via X-Powered-By). This lets attackers skip the fingerprinting step and go directly to CVE lookup. Suppressing the version number does not fix the underlying software but materially raises the bar for mass-exploitation worms.

Details

Observed headers: https://ausecurity.best/ : Server: nginx/1.22.1 http://ausecurity.best:8080/ : Server: nginx/1.25.5 https://blog.ausecurity.best/ : Server: Apache/2.4.68 (Debian) https://ticket.ausecurity.best/ : Server: nginx/1.22.1 ; X-Powered-By: PHP/8.2.33 http://ticket.ausecurity.best:8080/ : Server: Apache/2.4.68 (Debian) ; X-Powered-By: PHP/8.2.33

Nikto additionally flagged nginx/1.22.1 and nginx/1.25.5 as "outdated". We did NOT confirm any specific exploitable CVE in these specific versions during this engagement — reduction here is defence-in-depth, not a direct exploit.

Evidence stored under: evidence/t1-ports/stage2-services.nmap , evidence/t-finalprobe/nikto-*.txt .

Recommendation

1. nginx: add `server_tokens off;` globally in nginx.conf. For full version-string removal install nginx-extras + the Headers More module and `more_clear_headers 'Server';`.
2. Apache: add `ServerTokens Prod` and `ServerSignature Off` in the main config.
3. PHP: in php.ini set `expose_php = Off` to drop the X-Powered-By header.
4. Independently of the version-string change, plan an upgrade cycle: nginx mainline 1.27+, keep Apache on the current Debian security track, keep PHP on a maintained 8.3+ branch when available.

M6: WordPress Admin Username Enumeration via REST API and Author Redirect	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://blog.ausecurity.best/
References	• CWE-204 (Observable Response Discrepancy) • WordPress REST API users endpoint

Overview

The WordPress REST API at /?rest_route=/wp/v2/users and /?rest_route=/wp/v2/users/1 returns unauthenticated JSON that identifies the administrator account by username and slug (`admin`), along with the author permalink and gravatar hash. The `?author=1` → 301 redirect also leaks the slug. Both are standard precursors to targeted credential attacks (brute-force, password reuse, credential stuffing) against the known admin username.

Details

Confirmed enumeration vectors:

1. GET https://blog.ausecurity.best/?rest_route=/wp/v2/users → HTTP 200 JSON array containing {id:1, name:"admin", slug:"admin", link:"https://blog.ausecurity.best/index.php/author/admin/", avatar_urls:{..., gravatar_hash: "25e5434e5d0e1a62822d3670d4cf46e6b59a88908848f9bb1c91d11c819aa61b"}}
2. GET https://blog.ausecurity.best/?rest_route=/wp/v2/users/1 → HTTP 200 JSON with the same admin data.
3. GET https://blog.ausecurity.best/?author=1 → HTTP 301 Moved Permanently with Location containing "/author/admin/".
4. The direct `/wp-json/wp/v2/users` endpoint is blocked (404), but the `?rest_route=` fallback (which WordPress enables when permalinks are not configured for the API) is not — so the mitigation is incomplete.

The gravatar hash additionally allows an attacker to confirm the admin's registered email across the public Gravatar database if the user is a Gravatar member, extending the identity disclosure beyond just the username.

Evidence stored under: evidence/t6-wordpress/manual-probes.txt , evidence/t-api2/probe2.txt , evidence/t10-nuclei/results-focused.txt (nuclei "wp-user-enum" LOW).

Recommendation

1. In WordPress, filter the REST API `/wp/v2/users` endpoint to require `edit_users` capability for anonymous callers. Can be done with a `rest_authentication_errors` or `rest_endpoints` filter in a mu-plugin.
2. Disable the `?rest_route=` fallback by configuring permalinks and enforcing routing so the only REST entry point is `/wp-json/*`, which is already 404.
3. Disable WordPress author archive URLs (`?author=N` redirect) via a `template_redirect` hook that 404s unauthenticated requests.
4. Rename the admin account from "admin" to a non-guessable value. Add a secondary account as the user-facing writer with no admin rights.
5. Enforce 2FA on all WordPress admins (plugin: Two-Factor, Wordfence, etc.).
6. Confirm that the WordPress login page is also protected by rate limiting and/or a CAPTCHA.

M7: WordPress Uploads Directory Listing Enabled	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://blog.ausecurity.best/wp-content/uploads/
References	CWE-548 (Exposure of Information Through Directory Listing)

Overview

Apache serves a browsable directory index at /wp-content/uploads/ on the WordPress blog, listing the yearly/monthly folders that WordPress creates and (recursively) every file ever uploaded. Even if no sensitive files are currently indexed, this is a persistent reconnaissance vector that will leak draft media, private attachments, and backup artefacts the moment they are placed under /wp-content/uploads/.

Details

GET https://blog.ausecurity.best/wp-content/uploads/ returns HTTP 200 with an auto-generated Apache "Index of /wp-content/uploads" listing. Observed indexed child: `2026/` folder. The behaviour is confirmed by both the WPScan output and manual curl probes.

Evidence stored under: evidence/t6-wordpress/manual-probes.txt , evidence/t6-wordpress/wpscan.txt.

Recommendation

1. Disable Apache directory listings globally: in the main Apache config remove `Indexes` from the `Options` line, or add an explicit `Options -Indexes` directive.
2. Alternatively (less preferred) drop a `.htaccess` file in /wp-content/uploads/ containing `Options -Indexes` plus `IndexIgnore *`.
3. Verify that no media files under /wp-content/uploads/ contain sensitive customer data (invoices, exported reports). If any do, move them out of the public uploads path and serve them through an authenticated download endpoint.

M8: osTicket web.config File Exposed	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• https://ticket.ausecurity.best/web.config • http://ticket.ausecurity.best:8080/web.config
References	CWE-538 (Insertion of Sensitive Information into Externally-Accessible File)

Overview

The IIS/ASP.NET-style web.config shipped with osTicket is served as plain text and reveals the application's internal URL rewrite rules (e.g. /api/* → /api/http.php/ , */scp/apps/* → /scp/apps/dispatcher.php/, */pages/* → /pages/index.php/*). This is intended for IIS deployments only and does nothing on the current nginx + Apache stack other than help attackers map the application.

Details

GET https://ticket.ausecurity.best/web.config returns HTTP 200 with a 2,197-byte XML body containing the IIS URL Rewrite Module configuration. The directives disclose at least three internal dispatcher endpoints and the overall PHP routing structure. Directory browsing is disabled per the file; the file itself is not used by the running web servers (nginx and Apache do not process web.config), so deleting it has no functional impact.

Evidence stored under: evidence/t-webprobe/webprobe.txt , evidence/t10-nuclei/results-focused.txt (nuclei "web-config" LOW).

Recommendation

1. Delete /web.config from the osTicket web root; it is only used by IIS, not by nginx or Apache.
2. In nginx, add a global deny for `location ~* /web\.config$ { deny all; }` as defence-in-depth.
3. While cleaning up, also delete /setup/ (see critical finding) and any `*.ini`, `*.sql`, `README*`, `CHANGELOG*` left over from the install.

I1: Advertised Mailbox info@ausecurity.best Cannot Receive Mail	
Score	0.0 (Info)
Vector string	N/A
Target	ausecurity.best (contact page)
References	RFC 7505 (null MX)

Overview

The public contact page advertises "info@ausecurity.best" as a point of contact, but the domain publishes no MX record and no mail server is reachable, so mail sent to that address will bounce or silently drop. Prospective clients attempting to reach AuSecurity by email will fail.

Details

`dig +short MX ausecurity.best @1.1.1.1` returns empty. The address appears in /contact.html as a user-facing contact method. This is also related to the critical "domain spoofability" finding.

Recommendation

1. Either stand up a real mailbox for info@ausecurity.best (through Google Workspace, Microsoft 365, or any ESP) and publish correct MX / SPF / DKIM / DMARC (see the critical spoofability finding), or
2. Remove the info@ mailbox reference from the website and replace with the authenticated /api/ contact form only, plus publish an explicit null MX (`ausecurity.best. 300 IN MX 0 .`) per RFC 7505.

I2: CAA, AAAA, robots.txt, sitemap.xml and security.txt Not Published for Apex	
Score	0.0 (Info)
Vector string	N/A
Target	ausecurity.best
References	• RFC 6844 (CAA) • RFC 9116 (security.txt) • sitemap.org

Overview

Several standard, low-cost hygiene items are not published for the apex website: there is no Certificate Authority Authorization (CAA) record restricting which CA may issue certificates, no IPv6 (AAAA) record, no robots.txt, no sitemap.xml, and no /.well-known/security.txt disclosing a coordinated-disclosure contact.

Details

Confirmed with direct DNS and HTTP probes:

- CAA ausecurity.best → (none)
- AAAA ausecurity.best → (none)
- GET https://ausecurity.best/robots.txt → HTTP 404
- GET https://ausecurity.best/sitemap.xml → HTTP 404
- GET https://ausecurity.best/.well-known/security.txt → HTTP 404

Impact: these are low-impact items individually but collectively they signal to attackers that security hygiene has not been through a baseline hardening pass, and the missing security.txt specifically makes it harder for third-party researchers to responsibly report issues.

Evidence stored under: evidence/t3-email/dns.txt , evidence/t-misc/misc-probes.txt .

Recommendation

1. Publish a CAA record: `ausecurity.best. 300 IN CAA 0 issue "letsencrypt.org"` and `0 iodef "mailto:security@ausecurity.best"`.
2. Publish a security.txt at /.well-known/security.txt per RFC 9116, listing a monitored email, PGP key (optional), scope, and preferred languages.
3. Publish a robots.txt declaring (at minimum) crawling policy for /dashboard.html, /login.html, /consultation-success.html (though this must NOT be used as a substitute for proper access control — see the dashboard finding).

4. Publish a sitemap.xml for SEO purposes if desired.
5. Add AAAA records once the hosting supports IPv6 reachably.

I3: Hosting Jurisdiction Mismatch (OVH Singapore vs Advertised Sydney Address)	
Score	0.0 (Info)
Vector string	N/A
Target	• 51.79.166.148 • 51.79.167.208 • 51.79.166.209 (AS16276 OVH SAS / OVH Singapore PTE LTD)
References	Australian Privacy Act 1988 APP 8 (Cross-border disclosure of personal information)

Overview

The three VPS hosts serving the website, blog, and ticketing system are physically located in Singapore (OVH), not in Australia, despite the business presenting a Sydney, NSW postal address. For a GRC consultancy whose Australian clients may expect data residency to match the business's advertised jurisdiction, this is a disclosure that should be made in the privacy policy and in client agreements.

Details

WHOIS / reverse DNS confirms:

- 51.79.166.148 → vps-10f1e4c3.vps.ovh.ca (OVH Singapore)
- 51.79.167.208 → vps-d64a720c.vps.ovh.ca (OVH Singapore)
- 51.79.166.209 → vps-91287016.vps.ovh.ca (OVH Singapore)

Advertised business address: 63 Paddington St, Paddington NSW 2021, Australia.

Recommendation

1. Update the privacy policy / terms of service to clearly disclose the hosting country (Singapore) and the chain of OVH entities involved, as required under APP 8.
2. If client contracts assume Australian data residency, migrate hosting to an Australian region (OVH BHS/GRA do not cover AU; AWS ap-southeast-2 / Azure Australia East / Google asia-southeast4 would).
3. If migration is not planned, obtain written consent from clients for cross-border disclosure to Singapore.

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section